Lecture 1:

Introduction to DOM

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- What is DOM?
- DOM Tree Structure
- HTML vs DOM
- References.
- The End.

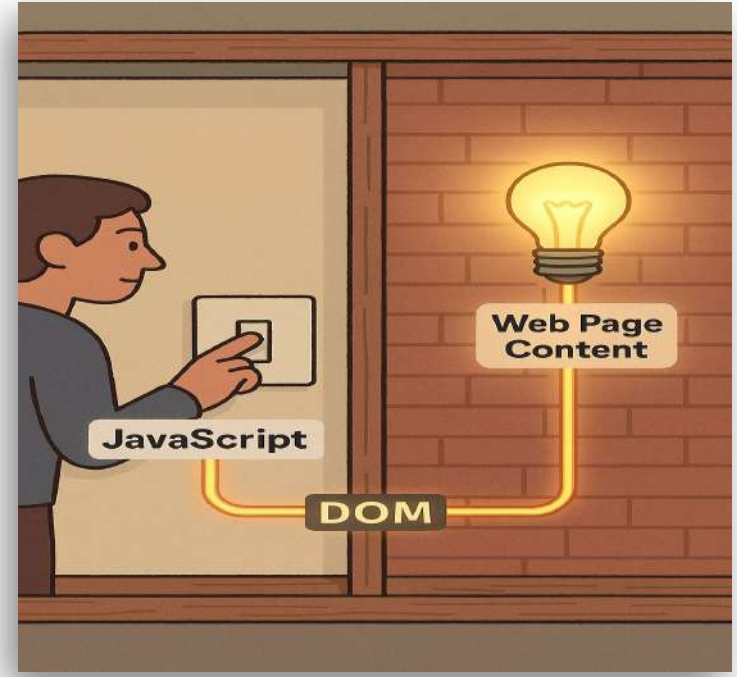
When you turn on the light using a switch, do you go inside the wall to move the wires yourself?

When you turn on the light using a switch, do you go inside the wall to move the wires yourself?

You don't touch the wiring directly — you use a switch, and the wiring does the rest to turn the light on or off.

The light bulb = **the content (HTML)**
The switch = **JavaScript**
The wiring = **the DOM**

The **DOM** is like the wiring in your house — it connects the switches (JavaScript) to the lights (web content), so things actually respond.

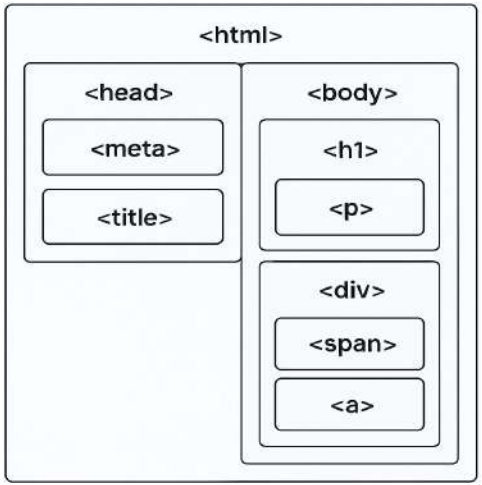


What is DOM?

DOM Stands for **Document Object Model.**

The DOM is a tree-like structure that the browser creates from your HTML page.

It represents everything on the webpage — text, buttons, images, sections — as objects that JavaScript can access and control.



Why We Need DOM?

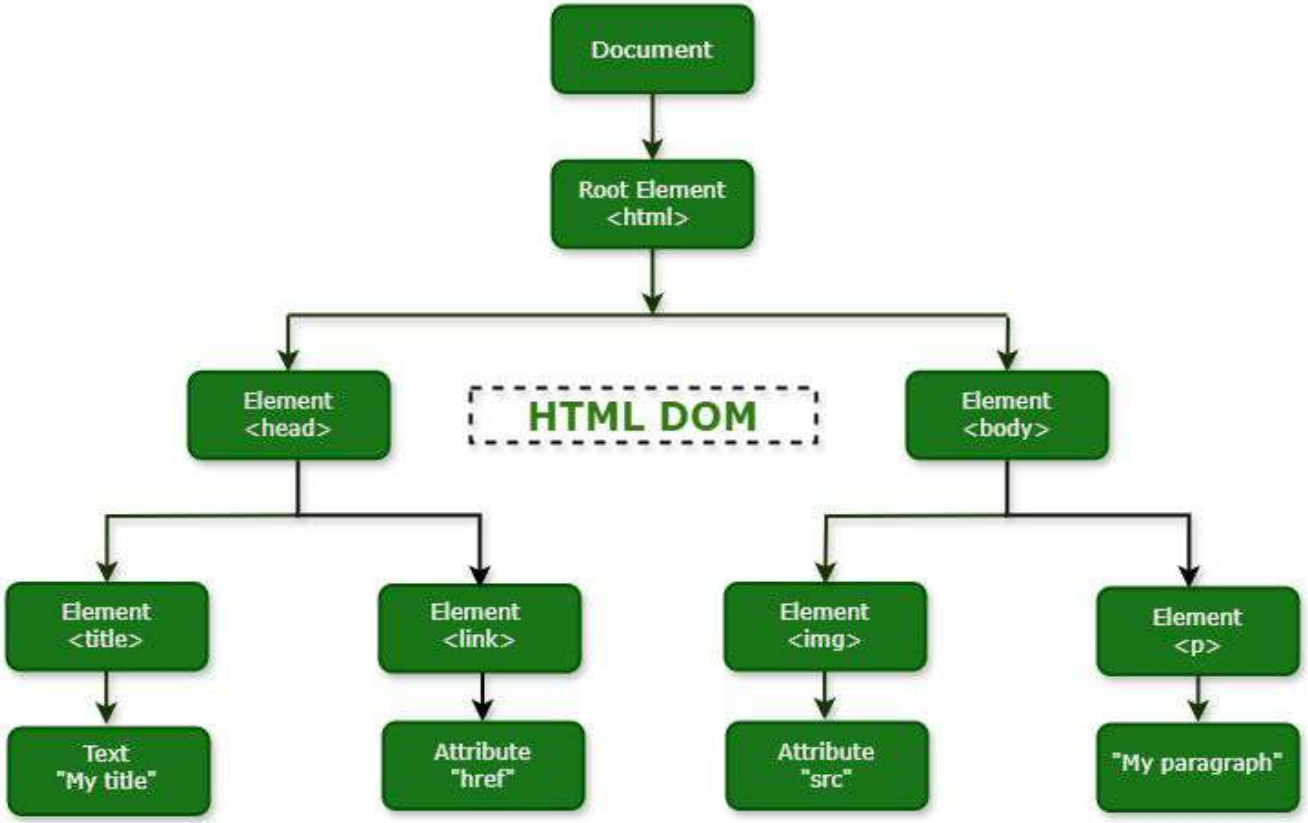
HTML creates the structure of a web page, but **once the page loads**, it's just static.
You can't change content, read input values, or respond to user actions with HTML alone.

Example: You can write a button in HTML, but without the DOM, JavaScript has no way to know when it's clicked or what to do after.

The DOM lets JavaScript:

- Read and change content on the page
- React to user actions (clicks, typing, scrolling)
- Create new elements or remove old ones
- Build dynamic, interactive web pages without reloading

DOM Tree Structure



DOM Tree Structure : Explanation

Root Level – Document Object

- The topmost node is **Document**.
- It represents the entire HTML page loaded in the browser.

<html> – Root Element

- The Document points to the **<html> element**, which is the root of the HTML structure.
- All content on the page is nested inside <html>.

Inside <html> – Two Main Branches

- <head>: Contains metadata and resources.
- <body>: Contains visible content of the page.

DOM Tree Structure

HEAD Section Breakdown

- **<head> → <title>**
  - <title> is a child of <head>.
  - It contains **text**: "My title" (this appears in the browser tab).

- **<head> → <link>**
  - <link> is another child of <head>.
  - It includes an **attribute**: "href" — typically used to link CSS files.

DOM Tree Structure

BODY Section Breakdown

- **‹body› → ‹img›**
  - ‹img› is a child of ‹body›.
  - It has an **attribute**: "src" — which defines the image source.

- **‹body› → ‹p›**
  - ‹p› is also a child of ‹body›.
  - It contains a **text node**: "My paragraph" — this appears as visible paragraph content.

DOM vs HTML

Although closely related, **HTML and DOM are not the same**. They serve different purposes in web development

Aspect	HTML	DOM
Type	Markup Language	Programming Interface
Nature	Static	Dynamic
Purpose	Structure and content	Manipulation and interaction
Modification	Cannot change after load	Can be changed at runtime
Used by	Browser	JavaScript
Event handling	Not supported	Fully supported

Check Your Understanding

Question : **What does DOM stand for in the context of web development?**

Options : a) Digital Ordinance Mandate

b) Document Object Model

c) Data Object Module

d) Document Orientation Marker

Answer : c) Document Object Model

Explanation: The DOM is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **JavaScript.info -** *A **comprehensive tutorial-style website** that covers core JavaScript concepts in depth.*
  🔗https://javascript.info/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 2:

DOM Traversal & Selection

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- DOM traversal
- getElementById()
- getElementsByClassName()
- References.
- The End.

Imagine you're in a room full of people. If you want to say something to *one specific person*, what's the first thing you do?

Imagine you're in a room full of people. If you want to say something to *one specific person*, what's the first thing you do?

You need to **identify** them — by name, by shirt color, or seat number.

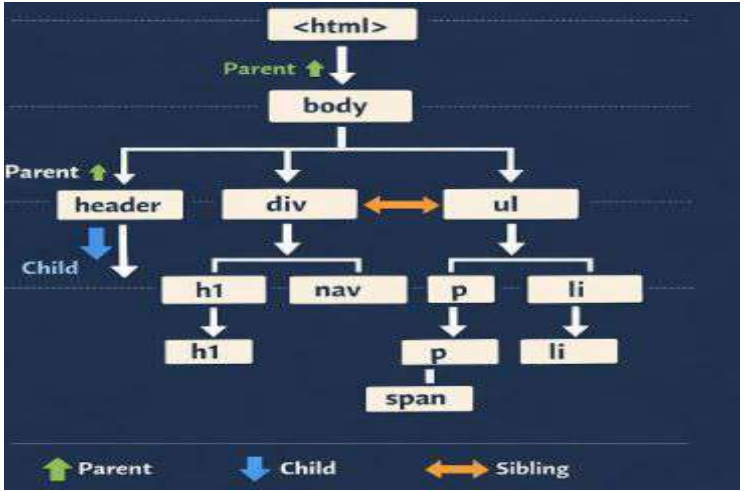
In the same way, JavaScript needs a way to **locate and select specific elements** on the page — like a button, heading, or paragraph — before it can interact with them.

That's why we use some selecting Elements in JS.

Introduction to DOM Traversal

DOM Traversal refers to the process of **navigating through the DOM tree** to access different nodes (elements, text nodes, or attributes) based on their **relationships** such as parent, child, and sibling.

Since an HTML document is represented as a **hierarchical tree structure**, DOM traversal enables developers to **move up, down, and sideways** within this structure to locate and manipulate specific elements dynamically.



Introduction to DOM Traversal

- Traversal works based on **node relationships**
- Element-based traversal avoids unwanted text nodes
- Widely used in frameworks and dynamic UI logic

```
const container = document.getElementById("container");

container.firstElementChild.style.color = "blue";
container.lastElementChild.style.fontWeight = "bold";
```

Selecting Elements

Selecting elements means accessing or grabbing specific parts of an HTML page (like a heading, button, paragraph, etc.) using **JavaScript**, so you can **read**, **change**, or **interact** with them.

Think of it like:

> ***"JavaScript is asking: Which part of the page do you want me to work with?"***

Selecting Elements

JavaScript gives us several built-in methods to select elements from the DOM. You choose which one to use based on *how* you want to target the element — by its ID, class, tag, or more advanced CSS-like selectors.

Some Common Built-in Methods :

- getElementById()
- getElementsByClassName()
- querySelector()
- querySelectorAll()

getElementById()

Selects **a single element** based on its id attribute.

IDs are **unique**, so this method always returns **one element** or null.

Syntax :

```
document.getElementById("elementId");
```

Note : **document** refers to the entire web page (DOM)

- It's the global object in the browser that represents your loaded HTML page.
- It's like JavaScript's entry point to interact with the webpage.

Think of document as the gatekeeper to the whole DOM tree.

getElementById()

Example :

```html
<h1 id="mainTitle">Welcome</h1>
```

HTML

```javascript
let heading = document.getElementById("mainTitle");
console.log(heading.textContent);  // Output: "Welcome";
```

JavaScript

Note : **.textContent** is a property that lets you **get or change the text inside an element** — we'll study this in detail later in this lecture.

getElementsByClassName()

Selects **all elements** that have a specific class name.

Returns an **HTMLCollection** (array-like, but not a real array).

Syntax :

```
document.getElementsByClassName("className");
```

- Returns a **live collection** of all matching elements.
- You need to loop through it for multiple elements.

getElementsByClassName()

Example :

HTML

```html
<p class="info">First</p>
<p class="info">Second</p>
```

JavaScript

```javascript
let paragraphs = document.getElementsByClassName("info");
console.log(paragraphs[0].textContent);  // Output: "First"
```

getElementById()vs getElementsByClassName()

Aspect	getElementById()	getElementsByClassName()
Selection	Single element	Multiple elements
Based on	id	class
Return type	Element	HTMLCollection
Live update	No	Yes
Speed	Faster	Slightly slower

Check Your Understanding

Question : **Which method is the most direct way to select a single HTML element that has a unique ID?**

Options : a) querySelector()

b) getElementsByClassName()

c) getElementById()

d) querySelectorAll()

Answer : c) getElementById()

Explanation: **getElementById()** is specifically optimized to find a single element by its unique ID, making it very fast and direct.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **JavaScript.info -** *A **comprehensive tutorial-style website** that covers core JavaScript concepts in depth.*
  🔗https://javascript.info/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 3:

Modern DOM Selectors

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- querySelector()
- querySelectorAll()
- NodeList vs HTMLCollection
- References.
- The End.

querySelector()

Selects the **first element** that matches a **CSS selector**.

More powerful and flexible than getElementById().

Syntax :

```
document.querySelector("selector");
```

- Returns **only the first matching element**.
- You can use **any CSS selector**: class (.card), id (#main), tag (div), etc.

querySelector()

Example :

HTML

```html
<div class="card">Hello</div>
```

JavaScript

```javascript
let card = document.querySelector(".card"); // Using CSS selector
console.log(card.textContent);  // Output: "Hello"
```

querySelector()

Example :

```
<p class="info">First paragraph</p>
<p class="info">Second paragraph</p>

//script.js

const para = document.querySelector(".info");
para.style.color = "blue";
```

Only the **first paragraph** is selected.

querySelector()

Example :

```
//Selecting by ID
const heading = document.querySelector("#title");

//Selecting by Class
const firstPara = document.querySelector(".text");

//Selecting by Tag
const paragraph = document.querySelector("p");

//Selecting Nested Elements
const listItem = document.querySelector("ul li");
```

querySelectorAll()

Selects **all elements** that match a **CSS selector**.

Returns a **NodeList** (can be looped using forEach()).

Syntax :

```
document.querySelectorAll("selector");
```

- Returns a **NodeList**, not live (unlike getElementsByClassName()).
- Supports full CSS selector syntax, and allows easy iteration.

querySelectorAll()

Example :

HTML

```html
<p class="note">One</p>
<p class="note">Two</p>
```

JavaScript

```javascript
let notes = document.querySelectorAll(".note");
notes.forEach(note => console.log(note.textContent));
// Output: "One", "Two"
```

NodeList vs HTMLCollection

Introduction

When selecting multiple elements from the DOM, JavaScript returns either a **NodeList** or an **HTMLCollection**. Both represent collections of nodes, but they differ in behavior and usage

NodeList vs HTMLCollection

NodeList

A **NodeList** is a collection of nodes returned by:

- querySelectorAll()
- childNodes

Characteristics

- Can include **element nodes and text nodes**
- Usually **static** (does not update automatically)
- Supports forEach() method
- Indexed like an array (but not an actual array)

NodeList vs HTMLCollection

HTMLCollection

An **HTMLCollection** is a collection of **HTML elements only**, returned by:

- getElementsByClassName()
- getElementsByTagName()
- children

Characteristics

- Contains **only element nodes**
- Always **live** (updates automatically when DOM changes)
- Does not support forEach() directly

Check Your Understanding

Question : **What is the main difference between *querySelector()* and querySelectorAll()?**

Options : a) **querySelector()** is newer and faster than **querySelectorAll()**.

b) **querySelector()** returns the first element that matches a CSS selector, while **querySelectorAll()** returns a list of all matching elements.

c) **querySelector()** can only use ID selectors, while **querySelectorAll()** can use class selectors.

d) **querySelector()** returns a live collection, while **querySelectorAll()** returns a static list.

Answer : b) **querySelector()** returns the first element that matches a CSS selector, while **querySelectorAll()** returns a list of all matching elements.

Explanation: You use **querySelector()** when you only need the first match, and **querySelectorAll()** when you need to access or iterate over all elements that match a selector.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **JavaScript.info -** *A **comprehensive tutorial-style website** that covers core JavaScript concepts in depth.*
  🔗https://javascript.info/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 4:

Styling DOM Elements

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- style property
- classList add/remove/toggle
- References.
- The End.

Now imagine you've found the person in the room. What if you want them to say something different? Or wear a different color jacket?

Now imagine you've found the person in the room. What if you want them to say something different? Or wear a different color jacket?

So after we select the element, the next step is to control what it says or how it looks — and that's exactly what we'll learn in: Changing Content & Styles.

If you want them to **say something else**, you change their **message** — like using .textContent or .innerHTML.

If you want them to **look different**, you change their **appearance** — like using .style or .classList.

Changing Content & Styles

Changing Content & Styles means using JavaScript to **update what an element says** (like text or HTML) and **how it looks** (like color, size, or layout) on the page.

Think of it like:

> ***"JavaScript is saying: Okay, now that I've found it — what should I make it say or how should it look?"***

Changing Content & Styles

JavaScript gives us built-in properties and methods to change the content and appearance of elements on the page.

You use these after selecting an element, depending on whether you want to update **text**, **HTML**, or **CSS styles**.

To Change Content:

- .innerText
- .innerHTML
- .textContent

To Change Styles:

- .style

.innerText

.innerText is a property in JavaScript that allows you to **get or change only the visible text** inside an HTML element.

It **ignores hidden content** (like elements styled with display: none or visibility: hidden), and respects line breaks and spacing as seen by the user.

Syntax :

```
element.innerText          // Get visible text
element.innerText = "New text here";  // Set visible text
```

.innerText

Example :

HTML

```html
<div id="info">
  Welcome Guest
</div>

```

JavaScript

```javascript
document.getElementById("info").innerText = "Hello, User!";
```

.innerText

Output:

Welcome Guest

Before Adding
.innerText

Hello, User!

After Adding
.innerText

.innerHTML

.innerHTML is a property that lets you **get or set the HTML content** (including tags and text) inside an element.

Unlike .innerText, .innerHTML reads and writes **HTML code**, not just plain text.

Syntax :

```
element.innerHTML          // Get HTML inside the element
element.innerHTML = "<b>Bold Text</b>";  // Set HTML inside the element
```

.innerHTML

Example :

HTML

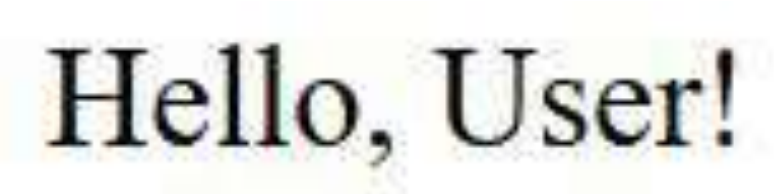
```html
<div id="box">Hello, User!</div>
```

JavaScript

```javascript
document.getElementById("box").innerHTML = "<strong>Welcome!</strong>";
```

.innerHTML

Output:



Before Adding
.innerHTML

After Adding
.innerHTML

.textContent

.textContent is a property in JavaScript that lets you **get or change the text** inside an HTML element.

It only affects **plain text**, not HTML tags.

Syntax :

```
element.textContent        // Get the text
element.textContent = "New text"  // Set or change the text
```

.textContent

Example :

HTML

```html
<h2 id="heading">Hello, User!</h2>
```

JavaScript

```javascript
document.getElementById("heading").textContent = "Welcome to the site!";
```

.textContent

Output:

Hello, User!

Before Adding
.textContent

Welcome to the site!

After Adding
.textContent

.textContent vs .innerText vs .innerHTML

Feature	.innerText	.textContent	.innerHTML
What it changes/gets	Only **visible text**	All text (even **hidden** ones)	Text **plus HTML tags**
Includes hidden elements?	No (ignores display: none)	Yes	Yes
Renders HTML tags?	No	No	Yes (interprets HTML)
Respects styling/layout?	Yes (includes line breaks etc.)	No	Yes
Use case	When you care about what's **visible**	When you want the **raw text**	When you want to **insert or get HTML code**
Performance	Slower (calculates layout)	Faster (reads DOM directly)	Medium (renders HTML)

Coding Questions

1. Simple Click Event

Task :

1. Given the HTML below, write a script that adds a 'click' event listener to the button. When the button is clicked, the text inside the paragraph (**<p>**) should change to "Button was clicked!".

Coding Questions

Solution :

HTML

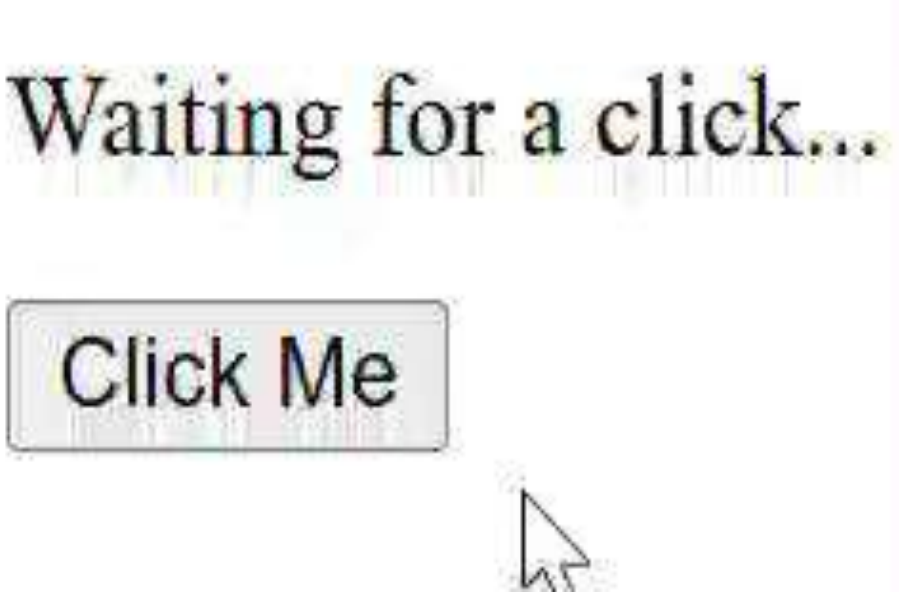
```html
<p id="status-text">Waiting for a click...</p>
<button id="action-button">Click Me</button>
```

JavaScript

```javascript
// 1. Select both elements
const statusText = document.getElementById('status-text');
const actionButton = document.getElementById('action-button');

// 2. Add the event listener to the button
actionButton.addEventListener('click', () => {
  // 3. This callback function runs on click
  statusText.textContent = "Button was clicked!";
});
```

Coding Questions

Output :



.style

.style is a property in JavaScript that allows you to **change the inline CSS styles** of an HTML element directly from your script.

Note: Use **camelCase** for multi-word CSS properties.

Example: background-color becomes backgroundColor.

Syntax :

```
element.style.propertyName = "value";
```

.style

Example :

HTML

```html
<div id="box">Hello</div>
```

JavaScript

```javascript
document.getElementById("box").style.color = "blue";
document.getElementById("box").style.backgroundColor = "lightgray";
```

.style

Output:

Before Adding .style

After Adding .style

Coding Questions

1. Select and Style

Task :

1. Given the HTML below, write a JavaScript snippet to select the paragraph element with the ID **main-text** and change its text color to **red** and its font size to **24px**.

Coding Questions

Solution :

HTML

```html
<p id="main-text">This is the target paragraph.</p>
```

JavaScript

```javascript
// 1. Select the element by its ID
const textElement = document.getElementById('main-text');

// 2. Change its style properties
textElement.style.color = 'red';
textElement.style.fontSize = '24px';
```

Coding Questions

Output :

This is the target paragraph.

Creating and Removing Elements

Creating elements means generating new items (like text, boxes, images, etc.) to be added to a web page or application interface.

Removing elements means deleting or taking out existing items from a web page so they are no longer visible or part of the layout.

Some Common Methods:

- .createElement();
- .appendChild();
- .removeElement();

.createElement()

document.createElement() is a JavaScript method used to **create a new HTML element** (like , <p>, <button>, etc.) in memory.

It doesn't appear on the page until you add it using a method like .appendChild().

Syntax :

```
document.createElement(tagName);
```

- **tagName:** A string representing the HTML tag you want to create (e.g., 'div', 'p', 'img').
- **Returns:** A new DOM element (but not yet on the page).

.appendChild()

.appendChild() is a DOM method used to **add an element as the last child** inside a parent element.

It places the new element **on the actual web page** (if it was created using .createElement() or already exists).

Syntax :

```
parentElement.appendChild(childElement);
```

- **parentElement:** The existing DOM element where you want to insert something.
- **childElement:** The element you want to insert at the end of parentElement.

Example: .createElement() and .appendChild()

HTML

```html
<!DOCTYPE html>
<html>
<head>
  <title>Create & Append with Event Listener</title>
</head>
<body>
  <h2>Shopping List</h2>
  <ul id="shopping-list">
    <li>Milk</li>
    <li>Eggs</li>
  </ul>
  <button id="add-btn">Add Bread</button>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript

```javascript
// Step 1: Select the button
const button = document.getElementById("add-btn");

// Step 2: Add event listener for 'click'
button.addEventListener("click", function () {
  // Step 3: Create new <li> element
  const newItem = document.createElement("li");
  newItem.textContent = "Bread";

  // Step 4: Append it to the <ul>
  const list = document.getElementById("shopping-list");
  list.appendChild(newItem);
});
```

Output : .createElement() and .appendChild()

.removeChild()

.removeChild() is a DOM method used to remove a child element from its parent in the HTML document.

You must call it **on the parent element** and pass the **child element you want to remove**.

Syntax :

```
parentElement.removeChild(childElement);
```

- **parentElement:** The DOM element that contains the child.
- **childElement:** The DOM element you want to remove.

Example: .removeChild()

HTML

```html
<!DOCTYPE html>
<html>
<head>
  <title>Remove with Event Listener</title>
</head>
<body>
  <h2>Shopping List</h2>
  <ul id="shopping-list">
    <li>Milk</li>
    <li>Eggs</li>
    <li>Bread</li>
  </ul>
  <button id="remove-btn">Remove Last Item</button>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript

```javascript
// Step 1: Select the button
const removeButton = document.getElementById("remove-btn");

// Step 2: Add event listener for 'click'
removeButton.addEventListener("click", function () {
  // Step 3: Select the <ul> and its last child
  const list = document.getElementById("shopping-list");
  const lastItem = list.lastElementChild;

  // Step 4: Remove the last item if it exists
  if (lastItem) {
    list.removeChild(lastItem);
  }
});
```

Output : .removeChild()

Introduction to classList

classList is a **property of DOM elements**

Used to manage CSS classes dynamically

Allows adding, removing, and toggling classes

Keeps CSS and JavaScript separate

Method	Purpose
add()	Adds class
remove()	Removes class
toggle()	Adds/removes class
contains()	Checks class

classList.add()

```
//style.css


.box {
  background-color: green;
  color: white;
  padding: 10px;
}

//script.js

document.getElementById("div1").classList.add("box");
```

classList.remove()

```
document.getElementById("div1").classList.remove("box");
```

Removes all styles applied by the class

classList.toggle()

```
document.getElementById("div1").classList.toggle("box");
```

How it works:

- If class exists → removes it
- If class does not exist → adds it

Check Your Understanding

Question : **Which property should you use to change the text inside a *<p>* tag without interpreting any HTML tags that might be in the string?**

Options : a) innerHTML

b) textContent

c) style

d) value

Answer : b) textContent

Explanation: ***textContent*** (and ***innerText***) treats all content as plain text. ***innerHTML***, on the other hand, will parse and render any HTML tags within the string, which can be a security risk if the content is from a user

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **JavaScript.info -** *A **comprehensive tutorial-style website** that covers core JavaScript concepts in depth.*
  🔗https://javascript.info/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 5:

Event Handling Basics

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- addEventListener()
- removeEventListener().
- Event object
- References.
- The End.

You're sitting comfortably at home. You're not randomly opening the door every minute, right?
So how do you know when to open it?

You're sitting comfortably at home. You're not randomly opening the door every minute, right?
So how do you know when to open it?

You **wait** until someone **knocks**.

The **knock** is your signal — only when you hear it, you get up and open the door.

Now think of JavaScript...

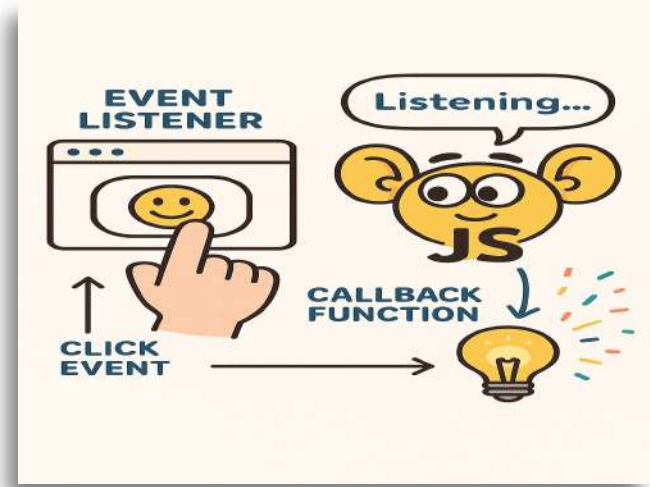
- Just like you listen for a knock, JavaScript can **listen for events** like clicks, scrolls, or keypresses.
- When the event happens, it **runs a function in response** — just like opening the door when someone knocks.

Event Listeners

An **event listener** is a way to make your web page **react to user actions** like clicking, typing, hovering, scrolling, etc.

It **"listens"** for a specific event on an element, and when that event happens, it runs the function (or code) you tell it to.

We use a special built-in method called **addEventListener()** to *add a listener* to any element on the page.



addEventListener()

.addEventListener() is a built-in JavaScript method that lets you **listen for specific events** (like clicks, keypresses, mouse movements, etc.) on an element — and then **run some code** when that event happens.

Syntax :

```
element.addEventListener("eventType", callbackFunction);
```

- element – The HTML element you want to attach the listener to.
- "eventType" – A string like "click", "keydown", "submit", etc.
- callbackFunction – The function that runs when the event happens.

addEventListener()

Example :

HTML

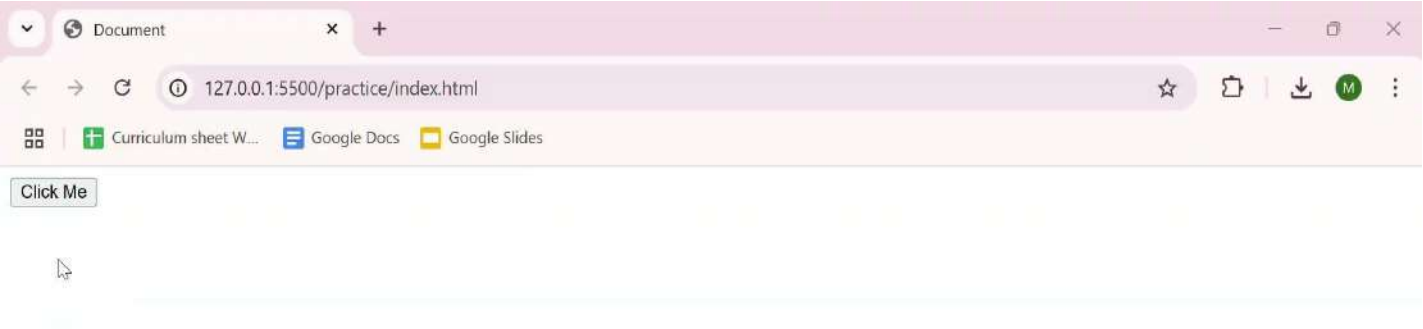
```html
<button id="btn">Click Me</button>
```

JavaScript

```javascript
document.getElementById("btn").addEventListener("click", function () {
  alert("Button was clicked!");
});
```

addEventListener()

Output :



removeEventListener()

removeEventListener() is used to **remove a previously attached event listener** from an element. This is useful when an event handler is no longer needed, helping to improve **performance and memory management**.

```
element.removeEventListener(eventType, eventHandler);
```

The same function reference used in addEventListener() must be passed to remove it.

removeEventListener()

Example

```
//index.html
<button id="btn">Click Me</button>
<button id="stop">Stop</button>

//script.js
const button = document.getElementById("btn");
const stopBtn = document.getElementById("stop");

function showMessage() {
  alert("Button clicked");
}

button.addEventListener("click", showMessage);

stopBtn.addEventListener("click", function () {
  button.removeEventListener("click", showMessage);
});
```

After clicking **Stop**, the button will no longer respond.

Event Object

The **Event Object** is an object automatically passed to the event handler when an event occurs.
It contains detailed information about the event such as:

- Type of event
- Target element
- Mouse position
- Keyboard key pressed
- Default behavior

```
element.addEventListener("click", function (event) {
  console.log(event);
});
```

Common Properties of Event Object

Property	Description
event.type	Type of event (click, keyup, submit)
event.target	Element that triggered the event
event.currentTarget	Element on which listener is attached
event.clientX	Mouse X position
event.clientY	Mouse Y position
event.key	Key pressed (keyboard events)
event.preventDefault()	Prevents default behavior
event.stopPropagation()	Stops event bubbling

Check Your Understanding

Question : **What are the two most essential arguments passed to the *addEventListener()* method?**

Options : a) The element and the style to change.

b) The event type (e.g., 'click') and the listener (callback) function to run.

c) The function name and a boolean value.

d) The mouse coordinates and the key that was pressed.

Answer : b) The event type (e.g., 'click') and the listener (callback) function to run.

Explanation: You must tell ***addEventListener()*** *what* to listen for (the event type string) and *what* to do when it happens (the listener/callback function).

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **JavaScript.info -** *A **comprehensive tutorial-style website** that covers core JavaScript concepts in depth.*
  🔗https://javascript.info/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 6:

Event Types

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- click & mouseover
- keydown & keyup.
- submit events
- References.
- The End.

Yesterday we learned how to listen for a 'click'. But users do much more — they type, scroll, move the mouse, submit forms, and more. Let's dive into the wide world of DOM events.

Types of Events

JavaScript events can be categorized into several groups.

Some Common Categories :

- Mouse Events.
- Keyboard Events.
- Form Events.

Mouse Events

Mouse events are a type of browser event that occur when the user interacts with the mouse — clicking, hovering, moving, or pressing mouse buttons on elements in a webpage.

These events allow JavaScript to **respond to user actions** and make the web page interactive.

Common Mouse Events :

- click.
- mouseover.

click Event

The click event fires when the user **presses and releases** the left mouse button on an element.

Commonly used to trigger actions like form submission, opening modals, toggling UI.

Syntax :

```
element.addEventListener('click', function() {
  // Code to run when element is clicked
});
```

Example : click Event

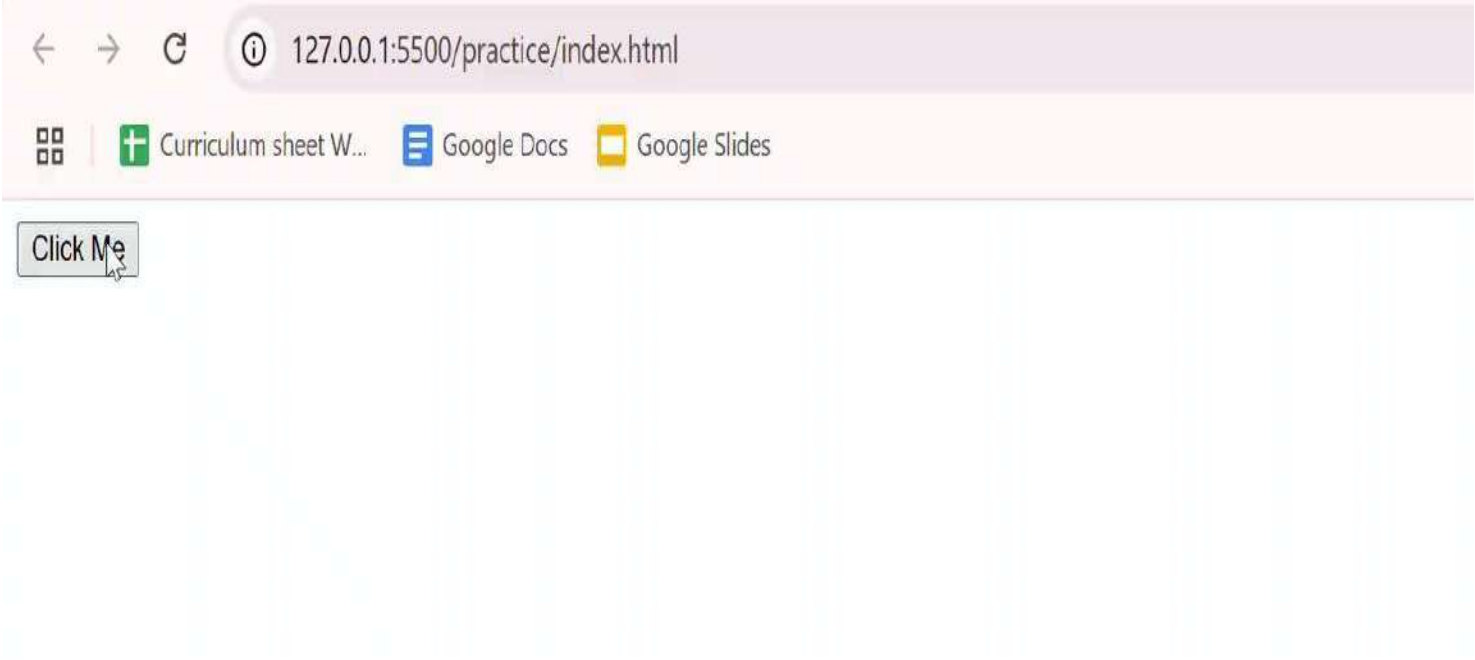
HTML

```html
<button id="myButton">Click Me</button>
```

JavaScript

```javascript
// JavaScript to handle click event
document.getElementById('myButton').addEventListener('click', function() {
  alert('Button clicked!');
});
```

Output : click Event



mouseover Event

The mouseover event fires when the mouse **enters** the area of an element.

Every time the pointer **enters** the element or **any child element** of it.

Syntax :

```
element.addEventListener('mouseover', function() {
  // Code to run when mouse hovers over the element
});
```

Example : mouseover Event

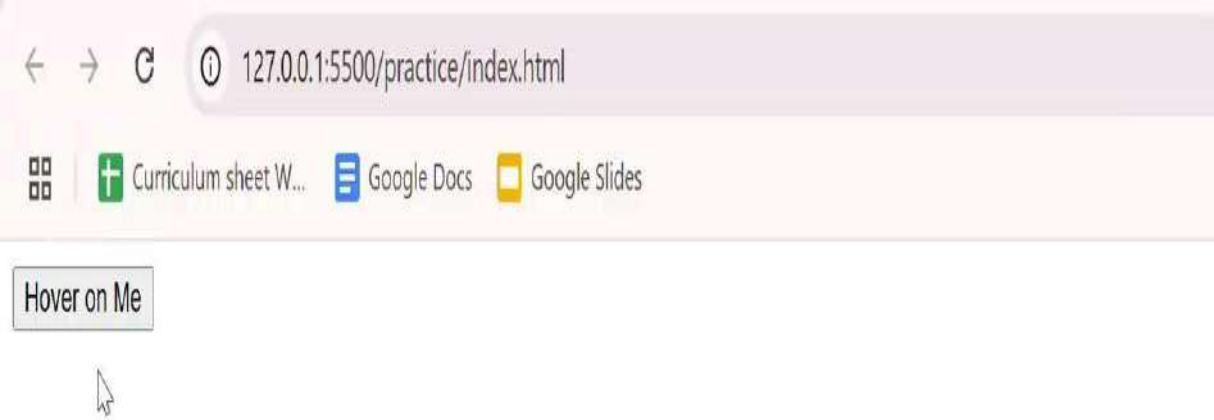
HTML

```html
<button id="myButton">Hover on Me</button>
```

JavaScript

```javascript
document.getElementById('myButton').addEventListener('mouseover', function() {
    alert('hovered on Button!');
});
```

Output : mouseover Event



Coding Questions

1. *mouseover* and *mouseout* Events

Task :

1. Given the HTML below, write a script to select the **div** with the ID **hover-box**. Add an event listener so that when the mouse moves over it, its background color changes to **lightblue**. Add another event listener so that when the mouse moves out, the background color reverts to **white**.

Coding Questions

Solution :

HTML

```html
<div id="hover-box"></div>
```

CSS

```style
<style>
#hover-box {
  width: 200px;
  height: 200px;
  border: 1px solid black;
}
</style>
```

JavaScript

```javascript
const hoverBox = document.getElementById('hover-box');

// Listener for when the mouse enters the element
hoverBox.addEventListener('mouseover', () => {
  hoverBox.style.backgroundColor = 'lightblue';
});

// Listener for when the mouse leaves the element
hoverBox.addEventListener('mouseout', () => {
  hoverBox.style.backgroundColor = 'white';
});
```

Coding Questions

Output :

Keyboard Events

A **keyboard event** in JavaScript is triggered when the user **presses**, **holds**, or **releases** a key on the keyboard.

It allows your webpage to **respond to key actions**, like shortcuts, search input, form navigation, games, etc.

Common Keyboard Events :

- keydown.
- keyup.

keydown Event

The keydown event is triggered **when a key is pressed down** on the keyboard.

It fires **once**, immediately when the key is pressed (even if it's held down).

Syntax :

```
element.addEventListener('keydown', function(event) {
  // Code to run when a key is pressed
});
```

Example : keydown Event

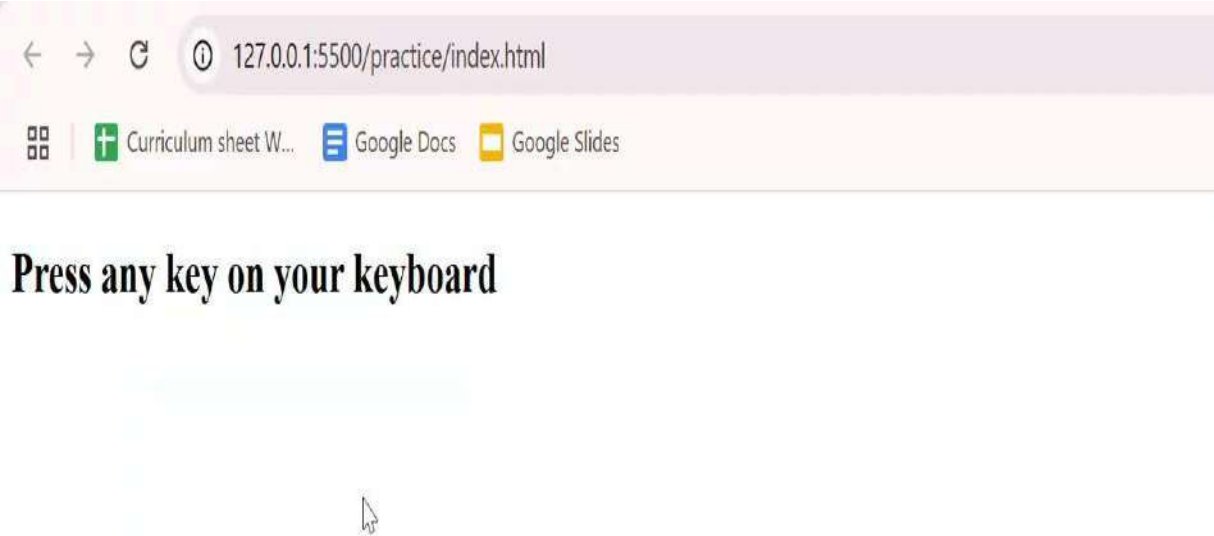
HTML

```html
<h2>Press any key on your keyboard</h2>
```

JavaScript

```javascript
// Listen for any key press on the whole document
document.addEventListener('keydown', function(event) {
      alert("You pressed: " + event.key);
});
```

Output : keydown Event



keyup Event

The keyup event is triggered **when a key is released** after being pressed.

It's useful when you want to perform an action **after** the keypress is complete.

Syntax :

```
element.addEventListener('keyup', function(event) {
  // Code to run when the key is released
});
```

Example : keyup Event

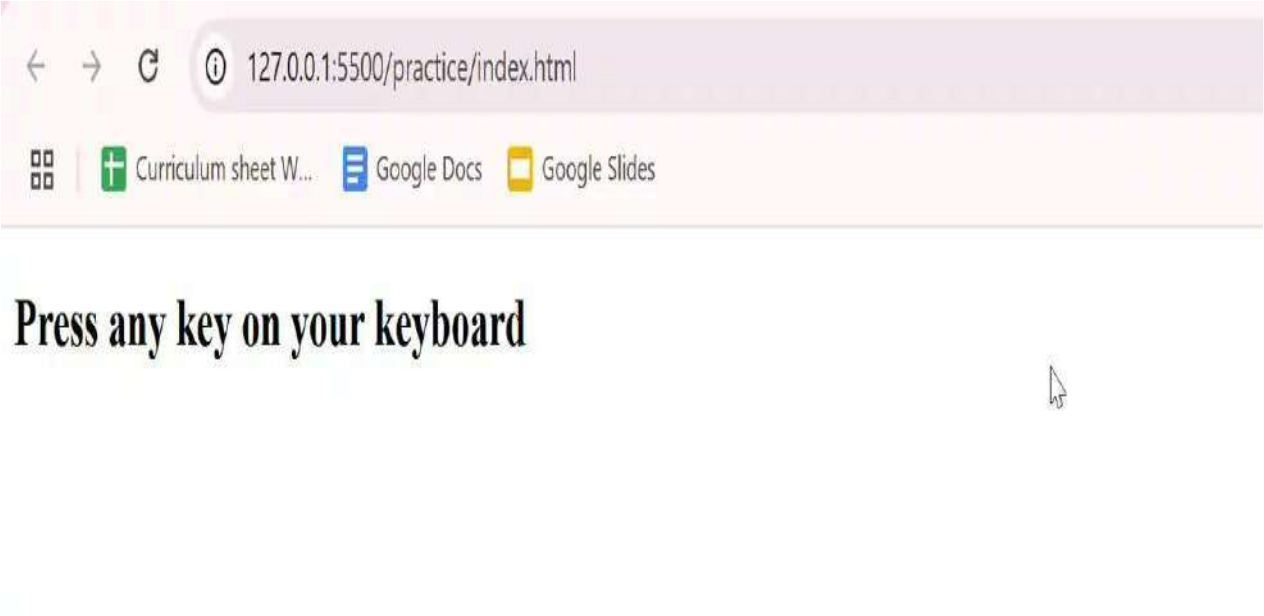
HTML

```html
<h2>Press any key on your keyboard</h2>
```

JavaScript

```javascript
document.addEventListener('keyup', function(event) {
        alert("You released a key : ", event.key);
});
```

Output : keyup Event



Coding Questions

1. *keydown* Event

Task :

1. Given the HTML below, add a **keydown** event listener to the input field. When the user presses any key while the input is focused, log the **key** property from the **event** object to the console.

Coding Questions

Solution :

HTML

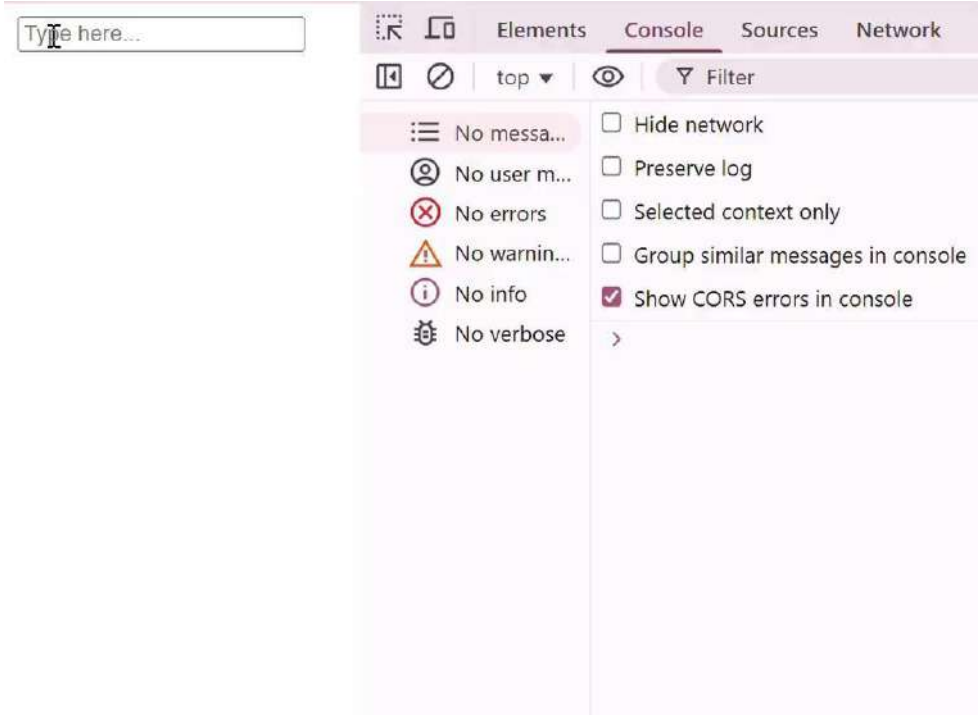
```html
<input type="text" id="key-input" placeholder="Type here..."/>
```

JavaScript

```javascript
const keyInput = document.getElementById('key-input');

keyInput.addEventListener('keydown', (event) => {
  // The 'event' object contains information about the event
  console.log(`Key pressed: ${event.key}`);
});

```

Coding Questions

Output :



Forms Events

Form events are triggered by **user interactions** with HTML form elements like <input>, <textarea>, <select>, and <form> itself.

JavaScript can **listen** to these events to validate, modify, or control form behavior in real time.

Common Form Event :

- submit.

submit Event

The submit event occurs **when a form is submitted** — usually when the user clicks a submit button or presses **Enter** inside a form field.

It allows you to **handle or prevent** form submission using JavaScript, such as validating input before sending.

Syntax :

```
formElement.addEventListener('submit', function(event)
{
  // your code here
});
```

Example : submit Event

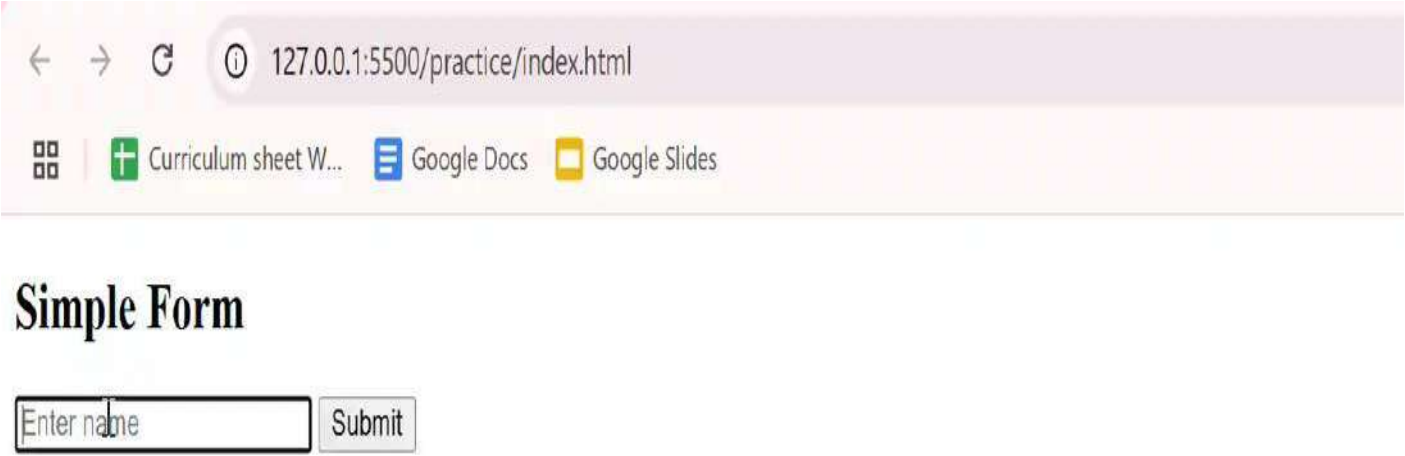
HTML

```html
<h2>Simple Form</h2>

<form id="myForm">
    <input type="text" placeholder="Enter name" />
    <button type="submit">Submit</button>
</form>
```

JavaScript

```javascript
const form = document.getElementById('myForm');

form.addEventListener('submit', function(event) {
    event.preventDefault(); // stop form from submitting normally
    alert("Form submitted!");
});
```

Output : submit Event



Check Your Understanding

Question : **Which event is fired when a user presses down on a key on the keyboard?**

Options : a) keyup

b) keypress

c) keydown

d) click

Answer : c) keydown

Explanation: The **keydown** event is triggered the moment a key is pressed down. **keyup** is fired when the key is released.

Check Your Understanding

Question : **Which event is triggered when the mouse pointer moves onto an element?**

Options : a) click

b) mouseout

c) mouseover

d) mousemove

Answer : c) mouseover

Explanation: **mouseover** fires once when the mouse enters the element's boundary. **mousemove** fires continuously as the mouse moves over the element.

Check Your Understanding

Question : **The *submit* event is typically used with which HTML element?**

Options : a) <button>

b) <input>

c)

d) <form>

Answer : d) <form>

Explanation: The **submit** event is fired on the **<form>** element itself when the user attempts to submit it, either by clicking a submit button or pressing Enter.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **JavaScript.info -** *A **comprehensive tutorial-style website** that covers core JavaScript concepts in depth.*
  🔗https://javascript.info/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 7:

Event Propagation

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

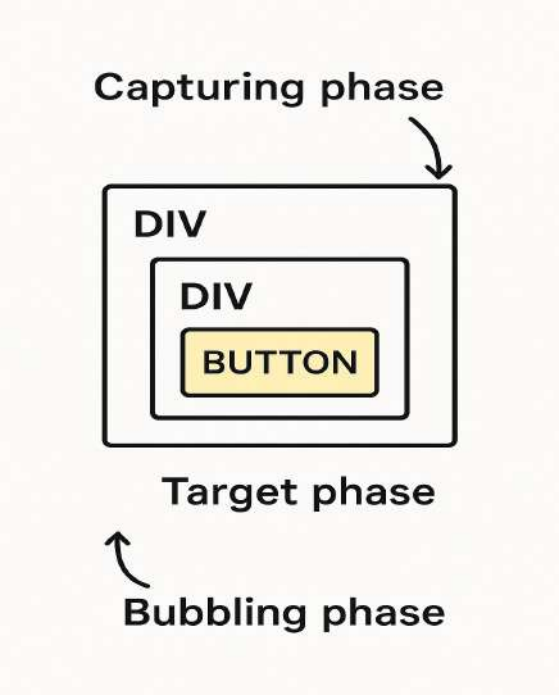
Table of Contents

- Bubbling
- Capturing
- preventDefault()
- References.
- The End.

If you have a button inside a div, and both have a click listener, which one runs first when you click the button?

If you have a button inside a div, and both have a click listener, which one runs first when you click the button?

The answer is 'it depends,' and it reveals a fundamental mechanism of how events travel through the DOM called **Event Propagation**."



Event Propagation

Event Propagation is the process that defines how events **travel through the DOM tree** when an event is triggered (like a click).

It determines **which elements** respond first — parent or child — and in **what order** they respond.

Event Propagation has 3 phases:

- **Capturing Phase:**
  Event moves *from the root (document)* down *towards the target element*.
- **Target Phase:**
  The event reaches the **exact element** that was interacted with.
- **Bubbling Phase:**
  Event then moves *back up* from the target to the root.

Event Capturing

Event travels from the **outermost parent** down to the **target element**.

Executed **first**, when useCapture is set to true in addEventListener.

Order: Document → HTML → Body → Parent → Child (Target)

Syntax :

```
1 element.addEventListener('click', handler, true);
```

Example : Event Capturing

HTML

```html
<div id="outer" style="padding: 30px; background-color: lightblue;">
  Outer Div
  <div id="inner" style="padding: 30px; background-color: lightgreen;">
    Inner Div
    <button id="btn">Click Me</button>
  </div>
</div>
```

JavaScript

```javascript
document.getElementById("outer").addEventListener("click", () => {
    alert("Outer Div (Capturing)");
  }, true); // true = capturing phase

  document.getElementById("inner").addEventListener("click", () => {
    alert("Inner Div (Capturing)");
  }, true); // true = capturing phase

  document.getElementById("btn").addEventListener("click", () => {
    alert("Button Clicked");
  });
```

Output : Event Capturing

Event Bubbling

Event starts at the **target element** and bubbles **up to its ancestors**.

This is the **default** behavior in JavaScript.

Order: Target → Parent → Body → HTML → Document

Syntax :

```
element.addEventListener('click', handler); // bubbling by default
```

Example : Event Bubbling

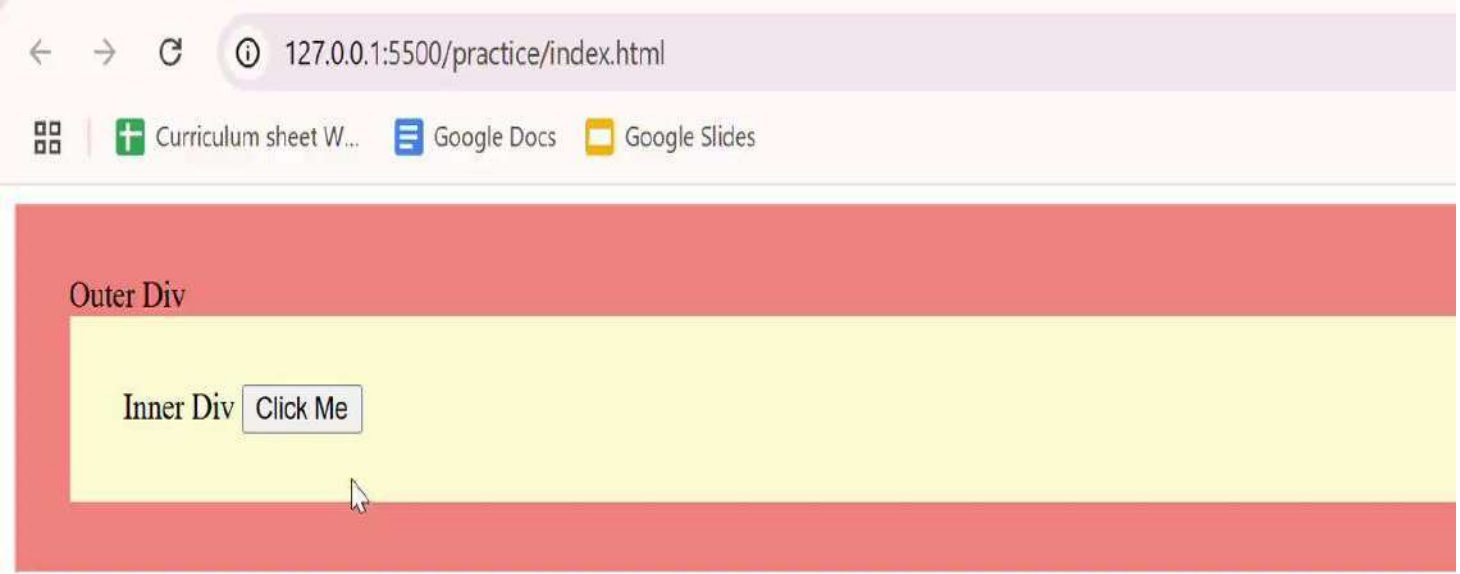
HTML

```html
<div id="outer" style="padding: 30px; background-color: lightcoral;">
  Outer Div
  <div id="inner" style="padding: 30px; background-color: lightgoldenrodyellow;">
    Inner Div
    <button id="btn">Click Me</button>
  </div>
</div>

```

JavaScript

```javascript
document.getElementById("outer").addEventListener("click", () => {
    alert("Outer Div (Bubbling)");
  });

  document.getElementById("inner").addEventListener("click", () => {
    alert("Inner Div (Bubbling)");
  });

  document.getElementById("btn").addEventListener("click", () => {
    alert("Button Clicked");
  });
```

Output : Event Bubbling



Some HTML elements have default behaviors — like ‹a› navigating or ‹form› refreshing. But what if we want JavaScript to handle those actions instead?

Some HTML elements have default behaviors — like <a> navigating or <form> refreshing. But what if we want JavaScript to handle those actions instead?

We need a way to tell the browser, 'Stop! Don't do your default action. I'll take it from here.'"

exactly what event.preventDefault() does — it stops the browser's automatic behavior so we can handle things our way using JavaScript.

When an event listener runs, it gets an Event object (e) with details about the event — like which key was pressed or where the mouse was clicked.

event.preventDefault()

A method on the event object that stops the browser from executing its default action for that event.

Syntax :

```
event.preventDefault();
```

Example : without event.preventDefault()

HTML

```html
<form id="myForm">
  <input type="text" />
  <button type="submit">Submit</button>
</form>
```

JavaScript

```javascript
document.getElementById("myForm").addEventListener("submit", function(e) {
  alert("Form submitted with Automatically");
});
```

Output : without event.preventDefault()

Example : with event.preventDefault()

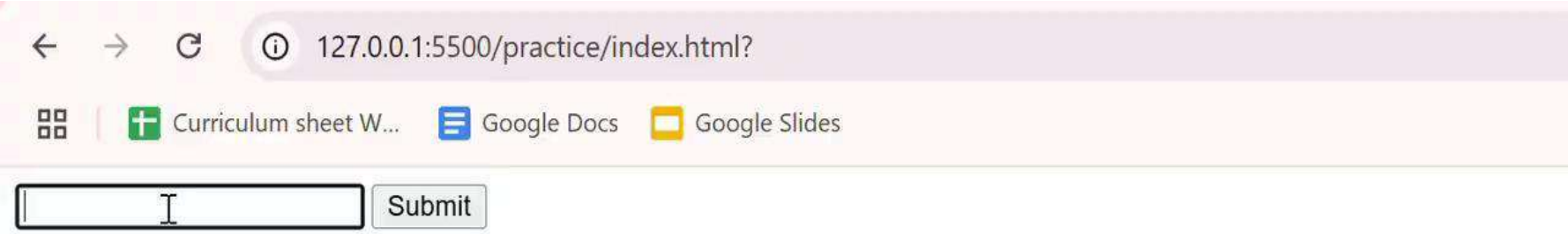
HTML

```html
<form id="myForm">
  <input type="text" />
  <button type="submit">Submit</button>
</form>
```

JavaScript

```javascript
document.getElementById("myForm").addEventListener("submit", function(e) {
    e.preventDefault(); // stops page reload
    alert("Form submitted with JavaScript!");
});
```

Output : with event.preventDefault()



Check Your Understanding

Question : **In modern browsers, what is the default phase of event propagation?**

Options : a) Capturing

b) Targeting

c) Bubbling

d) It is random

Answer : c) Bubbling

Explanation: By default, event listeners respond during the bubbling phase, where the event "bubbles up" from the target element to its ancestors.

Check Your Understanding

Question : **During the event bubbling phase, where does the event "travel"?**

Options : a) From the **window** object down to the target element.

b) From the target element up to its ancestors (parent, grandparent, etc.).

c) Only on the target element; it does not travel.

d) From the target element to its sibling elements.

Answer : b) From the target element up to its ancestors (parent, grandparent, etc.).

Explanation: Bubbling describes the process where an event, after firing on its target, propagates upwards through the DOM tree to parent elements.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **JavaScript.info -** *A **comprehensive tutorial-style website** that covers core JavaScript concepts in depth.*
  🔗https://javascript.info/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Lecture 8:

JS Execution Concepts

-By Instructor Name

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103

Table of Contents

- Execution Context
- Call Stack
- Temporal Dead Zone (TDZ)
- References.
- The End.

When you open a website and click a button, how does the browser remember which line of JavaScript code to execute next, and why does it sometimes throw an error even before running the full script?

When you open a website and click a button, how does the browser remember which line of JavaScript code to execute next, and why does it sometimes throw an error even before running the full script?

To answer this, we need to understand:

- **Execution Context** – where code runs
- **Call Stack** – how code execution is managed
- **Temporal Dead Zone (TDZ)** – why let and const behave strictly

Execution Context

An **Execution Context** is a **container or environment** created by the JavaScript engine to **run JavaScript code**.

Every time JavaScript runs:

- It creates an execution context
- Stores variables, functions, and scope information
- Executes code line by line

JavaScript **cannot execute code without an execution context**.

Why Execution Context Is Important

Execution context helps JavaScript:

- Know **which variables belong where**
- Decide **scope access**
- Assign the value of this
- Execute code in the correct order

Types of Execution Context

1. Global Execution Context (GEC)

- Created when the page loads
- Only one global context per page
- In browsers:
  - this refers to the **window object**
  - Global var variables become part of window

2. Function Execution Context (FEC)

- Created when a function is called
- Each function call creates a **new context**
- Has its own local memory and scope
- Destroyed after function execution

Execution Context Creation Phases

Phase 1: Memory Creation Phase (Hoisting)

During this phase:

- Memory is allocated for variables
- Function declarations are stored completely
- var → initialized as undefined
- let and const → hoisted but **not initialized**

No code is executed in this phase.

Execution Context Creation Phases

Phase 2: Code Execution Phase

During this phase:

- Code executes line by line
- Values are assigned to variables
- Functions are invoked

Execution Context

```
var a = 10;
let b = 20;

console.log(window.a);
console.log(window.b);
```

Output

10
undefined

Explanation

- a is part of global execution context and attached to window
- b exists in script scope, not global object

Function Execution Context – Example

```
function greet() {
  var message = "Hello User";
  console.log(message);
}

greet();
console.log(message);
```

Explanation

- message exists only inside function execution context
- Browser removes it after function execution

Call Stack

The **Call Stack** is a **stack-based data structure** used by the JavaScript engine to:

- Track function calls
- Maintain execution order
- Know where to return after execution

JavaScript uses a **single call stack**, meaning it can execute **only one task at a time**.

How Call Stack Works

1. Global execution context is pushed first
2. Function calls push new execution contexts
3. Completed functions are popped out

Global Context
→ first()
→ second()
→ third()
← third() ends
← second() ends
← first() ends

Call Stack Trace

```
function first() {
  second();
}

function second() {
  third();
}

function third() {
  console.trace();
}

first();
```

Browser prints the **actual call stack**

Call Stack Execution Order Example

```javascript
function main() {
  console.log("Main Start");
  helper();
  console.log("Main End");
}

function helper() {
  console.log("Helper Running");
}

main();
```

Main Start
Helper Running
Main End

Execution follows strict LIFO order

Call Stack Overflow (Important Concept)

```
function infinite() {
  infinite();
}

infinite();
```

Browser Error

Uncaught RangeError: Maximum call stack size exceeded

Why This Happens

- Call stack has limited memory
- Infinite calls fill the stack
- Browser stops execution to prevent crash

Temporal Dead Zone (TDZ)

The **Temporal Dead Zone (TDZ)** is the **time period between variable hoisting and initialization** for let and const.

During TDZ:

- Variable exists in memory
- Accessing it causes a **ReferenceError**

Purpose of TDZ

- Prevents bugs caused by accidental variable access
- Forces developers to declare variables before use
- Encourages predictable code behavior

TDZ vs var

Feature	var	let	const
Hoisted	Yes	Yes	Yes
Initialized	undefined	❌	❌
TDZ	❌	✅	✅
Block scope	❌	✅	✅

TDZ vs var

Feature	var	let	const
Hoisted	Yes	Yes	Yes
Initialized	undefined	❌	❌
TDZ	❌	✅	✅
Block scope	❌	✅	✅

```
console.log(score);
let score = 50;
```

Uncaught ReferenceError: Cannot access 'score' before initialization

Function-Level TDZ

```
let total = 100;

function calculate() {
  console.log(total);
  let total = 50;
}

calculate();
```

Explanation

- Local total shadows global
- Local variable is in TDZ
- Access causes error

Check Your Understanding

Question : **What is an Execution Context in JavaScript?**

Options : a) A file where JavaScript code is stored

b) An environment where JavaScript code is executed

c) A browser tab

d) A JavaScript library

Answer : b) An environment where JavaScript code is executed

Explanation: Execution context provides the environment needed to run JavaScript code.

Check Your Understanding

Question : **In a browser, what does this refer to in the global execution context?**

Options : a) document

b) window

c) global

d) undefined

Answer : b) window

Explanation: In browsers, the global execution context binds `this` to the `window` object.

Check Your Understanding

Question : **What data structure does JavaScript use to manage execution contexts?**

Options : a) Queue

b) Tree

c) Heap

d) Stack

Answer : d) Stack

Explanation: JavaScript uses a call stack to manage execution contexts.

Check Your Understanding

Question : **Accessing a variable in TDZ results in:**

Options : a) undefined

b) null

c) SyntaxError

d) ReferenceError

Answer : d) ReferenceError

Explanation: Accessing TDZ variables throws ReferenceError.

References

- **MDN Web Docs :** *Comprehensive and official documentation for JS.*
  🔗https://developer.mozilla.org/en-US/docs/Web/JavaScript

- **W3Schools – JS Tutorial :** *Beginner-friendly JS tutorials with examples and interactive editors.*
  🔗 https://www.w3schools.com/js/

- **JavaScript.info -** *A **comprehensive tutorial-style website** that covers core JavaScript concepts in depth.*
  🔗https://javascript.info/

Thanks For Watching

K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Course - Web Development - II

welcome@krmangalam.edu.in

Sohna Road,Gurugram,Haryana 122103